

```
In [1]: import pandas as pd
import numpy as np
from collections import Counter
import random
import matplotlib.pyplot as plt
from scratch.linear_algebra import Vector, vector_sum, dot
import tqdm
import math
import seaborn as sns
from scratch.gradient_descent import gradient_step
```

```
In [2]: # Import Data
df = pd.read_csv('persetujuan pinjaman.csv')
print (f"columns\t:{df.shape[1]}\nrows\t:{df.shape[0]}")
```

```
columns :6
rows    :100
```

```
In [3]: #EDA
#cleaning
df.columns=df.columns.str.lower()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   no              100 non-null   int64
 1   usia            100 non-null   int64
 2   pendapatan      100 non-null   int64
 3   skor_kredit     100 non-null   int64
 4   rasio_hutang    100 non-null   int64
 5   status          100 non-null   int64
dtypes: int64(6)
memory usage: 4.8 KB
```

```
In [4]: df.head ()
```

```
Out[4]:
```

	no	usia	pendapatan	skor_kredit	rasio_hutang	status
0	1	22	45	580	45	0
1	2	25	50	600	40	0
2	3	28	55	620	38	0
3	4	30	60	650	35	1
4	5	32	65	680	30	1

```
In [5]: # simple analist
df_=df[:]
df_.describe()
```

Out[5]:

	no	usia	pendapatan	skor_kredit	rasio_hutang	status
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
mean	50.500000	35.730000	71.610000	707.900000	28.500000	0.750000
std	29.011492	8.501759	17.358312	82.441225	9.433981	0.435194
min	1.000000	21.000000	42.000000	560.000000	13.000000	0.000000
25%	25.750000	28.750000	57.000000	640.000000	20.750000	0.750000
50%	50.500000	36.000000	72.000000	707.500000	27.000000	1.000000
75%	75.250000	42.250000	85.250000	772.500000	36.250000	1.000000
max	100.000000	52.000000	110.000000	850.000000	48.000000	1.000000

In [6]:

```
# scaling data
columns_=df_.columns[:-1]
for i in columns_:
    values=[]
    min=df_[i].min()
    max =df_[i].max()
    for j in range (len(df_['usia'])):
        x = df_.loc[j,i]
        value=(x-min)/(max-min)
        values.append(value)
    df_[i]=values
```

In [7]: df_.head()

Out[7]:

	no	usia	pendapatan	skor_kredit	rasio_hutang	status
0	0.000000	0.032258	0.044118	0.068966	0.914286	0
1	0.010101	0.129032	0.117647	0.137931	0.771429	0
2	0.020202	0.225806	0.191176	0.206897	0.714286	0
3	0.030303	0.290323	0.264706	0.310345	0.628571	1
4	0.040404	0.354839	0.338235	0.413793	0.485714	1

In [8]:

```
# Build split data
def setData (xs:list[Vector]):
    return [[1]+x for x in xs]

def shuffle_ (indx:list[int],prob:float):
    random.shuffle(indx)
    cut= int(prob*len(indx))
    return indx[cut:], indx[:cut]

def split_train_test(xs:list[Vector], ys:list[float], test_prob : float):
    train_prob=1-test_prob
    idx =[i for i in range (0,len(ys))]
    idx_train,idx_test =shuffle_ (idx,train_prob)
    return (
        [xs.iloc[i] for i in idx_train],
        [ys.iloc[i] for i in idx_train],
        [xs.iloc[i] for i in idx_test],
        [ys.iloc[i] for i in idx_test])
```

```
)
```

```
In [9]: # =====
# Build Model
# =====

def logistic (x:list[float]):
    predict=1.0/(1+math.exp(-x))
    return predict

def negative_gradient(xs:list[Vector],ys:list[float],beta):
    return vector_sum ([list_gradient(x,y,beta) for x,y in zip(xs,ys) ])

def list_gradient(xs:Vector,ys:int, beta:Vector):
    return [gradient_j(xs,ys,j,beta) for j in range(len(beta))]

def gradient_j(xs:Vector,ys:float,j:int,beta:Vector):
    return -(ys-logistic(dot(xs,beta)))*xs[j]
```

```
In [10]: def fit_logistic (xs:list[Vector],ys=list[float],epochs=2000, learning_ra
guess=[random.random() for _ in range (len(xs[0]))]
with tqdm.trange (epochs)as t:
    for epoch in t:
        gradient=negative_gradient(xs,ys,guess)
        guess = gradient_step(guess,gradient,-learning_rate)
        t.set_description (f"beta: {guess}")
    return guess
```

```
In [11]: # intercept
def inter(xs:list[Vector]):
    new_x=[[1]+x for x in xs ]
    return new_x
xs=[[2,2],[2,3]]
inter (xs)
```

```
Out[11]: [[1, 2, 2], [1, 2, 3]]
```

```
In [12]: def fungsi_(y_, y_predict):
    tp=tn=fp=fn=0

    for y_true, y_pred in zip(y_, y_predict):
        if y_true==1 and y_pred==1:
            tp+=1
        elif y_true==0 and y_pred==0:
            tn+=1
        elif y_true==0 and y_pred==1:
            fp+=1
        elif y_true==1 and y_pred==0:
            fn+=1

    accuracy=(tp+tn)/(tp+tn+fn+fp) if (tp+tn+fn+fp)!=0 else 0
    presisi=tp/(tp+fp) if (tp+fp)!=0 else 0
    recall=tp/(tp+fn) if (tp+fn)!=0 else 0
    f1_score=2*((presisi*recall)/(presisi+recall)) if (presisi+recall)!=0

    return (accuracy,presisi,recall,f1_score)
```

```
In [13]: def fungsi__(ys_test:float,xs_test:list[Vector],beta:list[Vector]):
        y_predicts=[]

        for i in range (len(ys_test)):
            y_predict=logistic(dot(beta,xs_test[i]))
            if y_predict<=0.5:
                y_predicts.append(0)
            else :
                y_predicts.append(1)

        y_predicts=[float(y) for y in y_predicts]

        y_test =[]
        for j in ys_test:
            y_test.append(j)

        accuracy,presisi,recall,f1_score=fungsi_(y_test,y_predicts)
        return print (f"accuracy:{accuracy}\nPresisi:{presisi}\nrecall:{recall}")
```

```
In [14]: columns_=df_.columns[1:]
        xs=df_[columns_[:-1]]
        ys =df_[columns_[-1]]
        print (xs)

        xs_train,ys_train,xs_test,ys_test=split_train_test(xs,ys,0.25)
```

	usia	pendapatan	skor_kredit	rasio_hutang
0	0.032258	0.044118	0.068966	0.914286
1	0.129032	0.117647	0.137931	0.771429
2	0.225806	0.191176	0.206897	0.714286
3	0.290323	0.264706	0.310345	0.628571
4	0.354839	0.338235	0.413793	0.485714
..	...	...	...	...
95	0.580645	0.529412	0.620690	0.285714
96	0.677419	0.632353	0.724138	0.228571
97	0.774194	0.735294	0.827586	0.171429
98	0.870968	0.823529	0.931034	0.085714
99	1.000000	1.000000	1.000000	0.000000

[100 rows x 4 columns]

```
In [15]: len(xs_test[0])
```

Out[15]: 4

```
In [16]: beta =fit_logistic (xs_train,ys_train,epochs=4000, learning_rate=0.0001)
```

```
0%|          | 0/4000 [00:00<?, ?it/s]/tmp/ipykernel_20799/328656450.p
y:17: FutureWarning: Series.__getitem__ treating keys as positions is depr
ecated. In a future version, integer keys will always be treated as labels
(consistent with DataFrame behavior). To access a value by position, use `
ser.iloc[pos]`
      return -(ys-logistic(dot(xs,beta)))*xs[j]
beta: [np.float64(1.219545497866212), np.float64(0.6468253677053352), np.f
loat64(1.53180995726148), np.float64(0.07486844388658324)]: 100%|
██████████| 4000/4000 [01:27<00:00, 45.56it/s]
```

```
In [17]: fungsi__(ys_test,xs_test,beta)
```

accuracy:0.72
Presisi:0.72
recall:1.0
f1_score:0.8372093023255813